



FLACSO
ECUADOR

SESIÓN 3-4: ETL CON POLARS

ETL con Polars: de la ingesta al análisis

Fausto Jácome Pérez
FLACSO Ecuador

Tenemos cinco tablas dispersas. ¿Cómo las cargamos, limpiamos, combinamos y resumimos para responder una pregunta?

¿Dónde estamos?

2

SESIÓN ANTERIOR

Corregimos tipos con `cast`.

Recodificamos con `pl.when()`.

Filtramos y seleccionamos columnas.

Hicimos el primer join entre personas y hogar.

3

ESTA SESIÓN

ETL completo: ingesta desde CSV, Excel y R. Transformaciones con `with_columns`. Reshaping. Joins múltiples. Agregaciones y window functions.

4

PRÓXIMA SESIÓN

SQL: las mismas operaciones en otro lenguaje, y por qué conviene conocer los dos.

El caso de hoy: un registro escolar ficticio con cinco tablas. Al final tendremos una tabla analítica que liga notas, alumnos, profesores y paralelos.

Ingesta de datos

Leer desde CSV, Excel y archivos de R

Cargar desde CSV y Excel

Desde CSV

```
import polars as pl

alumnos = pl.read_csv("datos/alumnos.csv")

# Especificar tipos al leer
alumnos = pl.read_csv(
    "datos/alumnos.csv",
    schema_overrides={
        "cod_estudiante": pl.String
    },
)
```

`pl.read_csv()` infiere tipos columna por columna. Si una columna mezcla números y texto, la infiere como `String`. Siempre verificar el schema con `.schema` después de cargar.

Desde Excel

```
# Hoja por nombre
notas = pl.read_excel(
    "datos/escuela.xlsx",
    sheet_name="notas",
)

# Primera hoja (por defecto)
df = pl.read_excel("datos/escuela.xlsx")
```

`pl.read_csv()` con parámetros no estándar

Algunos archivos CSV usan separadores distintos a la coma o encodings distintos a UTF-8:

```
# Separador @, encoding ASCII
df = pl.read_csv("viaje.csv", separator="@", encoding="ascii")

# Separador punto y coma, encoding Latin-1 (común en archivos de Europa y Latinoamérica)
df = pl.read_csv("datos.csv", separator=";", encoding="latin-1")

# Separador tabulación
df = pl.read_csv("datos.tsv", separator="\t")
```

El encoding por defecto de Polars es `utf8`. Los archivos generados con Excel en Windows suelen venir en `latin-1` o `cp1252`. Si aparecen caracteres extraños al leer, el encoding es el primer lugar donde buscar.

Cargar desde R

```
import polars as pl
import pyreadr

# .rds: un solo objeto R
resultado = pyreadr.read_r("datos/f1_personas.rds")
df = pl.from_pandas(resultado[None])

# .RData: puede contener varios objetos; se accede por nombre
resultado = pyreadr.read_r("datos/endi.RData")
df_personas = pl.from_pandas(resultado["f1_personas"])
df_hogar     = pl.from_pandas(resultado["f1_hogar"])
```

`pyreadr` convierte el archivo R a un DataFrame de pandas; `pl.from_pandas()` lo traduce a Polars.

Operaciones básicas

select, drop, rename, inspección y ordenamiento

`.select()` y `.drop()`

Quedarse con columnas específicas

```
df = alumnos.select([
    "cod_estudiante",
    "nombre",
    "ciudad",
    "paralelo",
])
```

Devuelve un DataFrame nuevo solo con esas cuatro columnas. Las demás desaparecen.

Descartar columnas específicas

```
df = alumnos.drop([
    "anio_nac",
    "mes_nac",
    "dia_nac",
])
```

Devuelve todas las columnas excepto las listadas. Útil cuando hay muchas columnas y solo se quieren eliminar unas pocas.

`.rename()`

Cambia el nombre de una o más columnas usando un diccionario. Las columnas no listadas conservan su nombre original.

```
df = alumnos.rename({
    "cod_estudiante": "id",
    "altura":         "altura_cm",
    "nombre":         "alumno",
})
```

`.rename()` no reordena columnas ni cambia su contenido. Es una operación de etiquetas puras: solo afecta los nombres.

El diccionario va en el sentido `"nombre_viejo": "nombre_nuevo"`. Si se invierte el orden, Polars lanza un error cuando el nombre viejo no existe en el DataFrame.

`.rename()` con función

Además de un diccionario, `.rename()` acepta una función que se aplica a **todos** los nombres de columna. Útil cuando hay muchas columnas con un patrón sistemático:

```
# Pasar todas las columnas a minúsculas de una sola vez
df = df_new_alumnos.rename(lambda col: col.lower())

# Encadenar: primero lowercase, luego renombrar la que quedó distinta
df = (
    df_new_alumnos
    .rename(lambda col: col.lower())
    .rename({"nombre_apellidos": "nombre"})
)
```

La lambda recibe el nombre de cada columna como string y devuelve el nombre nuevo. Polars la aplica a todas las columnas en una sola pasada, sin necesidad de listarlas.

El orden importa: si se encadena `.rename(lambda)` con `.rename(dict)`, el diccionario debe usar los nombres **ya transformados** por la lambda, no los originales.

Comparar columnas con `set`

Un `set` es una colección sin orden ni duplicados. La resta de conjuntos (`A - B`) devuelve los elementos que están en `A` pero no en `B` . Es una forma rápida de detectar diferencias entre esquemas:

```
cols_orig = set(df_alumnos.columns)
cols_nuevo = set(df_new_alumnos.columns)

# ¿Qué hay en el nuevo que no está en el original?
print(cols_nuevo - cols_orig)  # {'NOMBRE_APELLIDOS', 'COD_ESTUDIANTE', ...}

# ¿Qué hay en el original que no está en el nuevo?
print(cols_orig - cols_nuevo)  # {'nombre', 'cod_estudiante', ...}
```

Convertir `.columns` a `set` descarta el orden y los duplicados. Si la diferencia es vacía (`set()`), los esquemas coinciden exactamente. Útil antes de cualquier `concat` o `join` para detectar discrepancias antes de que generen errores silenciosos.

`.head()`, `.slice()` y `.tail()`

Herramientas de inspección rápida para ver una porción del DataFrame:

```
alumnos.head(3)      # primeras 3 filas

alumnos.tail(3)       # últimas 3 filas

# slice(offset, length): desde la posición 4, tomar 3 filas
alumnos.slice(4, 3)
```

`.head(n)` es equivalente a `.slice(0, n)`. El índice de `.slice()` empieza en 0: `.slice(4, 3)` devuelve las filas 4, 5 y 6.

`.sort()`

```
# Ascendente (por defecto)
alumnos.sort("edad")

# Descendente
alumnos.sort("edad", descending=True)

# Múltiples columnas: primero por paralelo, luego por nombre dentro de cada paralelo
alumnos.sort(["paralelo", "nombre"])

# Nulls al final en lugar de al inicio (comportamiento por defecto)
notas.sort("nota", nulls_last=True)
```

Polars coloca los valores `null` al inicio por defecto al ordenar de forma ascendente.

`with_columns`

Añadir y transformar columnas sin reemplazar el DataFrame

Concepto: `with_columns`

`.with_columns()` recibe una lista de expresiones y **agrega** columnas al DataFrame. Si el alias coincide con una columna existente, la reemplaza en ese lugar. El DataFrame original no se modifica.

```
# Agregar una columna nueva: el resultado tiene todas las columnas originales + edad_meses
df = alumnos.with_columns([
    (pl.col("edad") * 12).alias("edad_meses"),
])

# Reemplazar una columna existente: mismas columnas, pero edad ahora es Int16
df = alumnos.with_columns([
    pl.col("edad").cast(pl.Int16).alias("edad"),
])

# Varias columnas en un solo llamado: más eficiente que encadenar varios with_columns
df = alumnos.with_columns([
    (pl.col("edad") * 12).alias("edad_meses"),
    pl.col("sexo").cast(pl.Categorical).alias("sexo"),
])
```

with_columns + cast

Cambiar el tipo de una o más columnas en un solo paso:

```
df = alumnos.with_columns([
    pl.col("cod_estudiante").cast(pl.String),
    pl.col("sexo").cast(pl.Categorical),
    pl.col("edad").cast(pl.Int16),
    pl.col("altura").cast(pl.Float32),
])
```

Por defecto, `cast` falla en silencio si el valor no cabe en el tipo destino y devuelve `null`. Para detectar errores, usar `strict=True`:

```
pl.col("edad").cast(pl.Int8, strict=True) # lanza error si hay overflow
```


with_columns + operaciones de cadenas

El namespace `.str` contiene todas las operaciones sobre texto. Estas mismas operaciones se aplican antes del join para normalizar la columna `materia`:

```
df = alumnos.with_columns([
    # Quitar espacios al inicio y al final
    pl.col("nombre").str.strip_chars().alias("nombre"),
    # Reemplazar guión por espacio en nombres compuestos (San-Jose -> San Jose)
    pl.col("nombre").str.replace_all("-", " ").alias("nombre"),
    # Pasar a mayúsculas
    pl.col("ciudad").str.to_uppercase().alias("ciudad"),
    # Eliminar tildes para normalizar texto antes de joins
    pl.col("nombre")
        .str.replace_all("á", "a")
        .str.replace_all("é", "e")
        .str.replace_all("í", "i")
        .str.replace_all("ó", "o")
        .str.replace_all("ú", "u")
        .alias("nombre_norm"),
])
```

str.contains() y regex básico

str.contains() devuelve un booleano por fila: True si el patrón se encuentra en el texto.

```
# Patrón fijo: contiene la palabra "negativo"
df.filter(pl.col("cuotas_pagadas").str.contains("negativo"))

# Alternativas con |: coincide si el valor es "si" o "SI" o "afirmativo"...
df.with_columns(
    pl.col("cuotas_pagadas")
        .str.contains(r"(?i)^(s[íi]|afirmativo|cumplido|positivo)$")
        .alias("cuotas_ok")
)
```

Símbolo	Significado
(?i)	Ignorar mayúsculas y minúsculas
^ / \$	Inicio / fin del texto
	O lógico: a b coincide con a o con b
[abc]	Cualquiera de los caracteres entre corchetes
(...)	Agrupar parte del patrón

with_columns + variables temporales

Los datos de nacimiento llegan en tres columnas separadas. `pl.date()` las combina en una fecha real que admite aritmética temporal:

```
df = alumnos.with_columns(  
    pl.date(pl.col("anio_nac"), pl.col("mes_nac"), pl.col("dia_nac")).alias("fecha_nac")  
)  
  
# Extraer componentes o calcular diferencias  
df = df.with_columns([  
    pl.col("fecha_nac").dt.year().alias("anio_nac_check"),  
    pl.col("fecha_nac").dt.weekday().alias("dia_semana_nac"),  
    (pl.date(2026, 1, 1) - pl.col("fecha_nac"))  
        .dt.total_days()  
        .alias("edad_dias"),  
])
```

`pl.date()` acepta expresiones de columna como argumentos. El resultado es de tipo `Date`, que permite restar fechas directamente y obtener una duración en días con `.dt.total_days()`.

with_columns + variables numéricas

Transformaciones aritméticas directas.

```
df = alumnos.with_columns([
    # Convertir altura de centímetros a metros
    (pl.col("altura") / 100)
    .round(2)
    .alias("altura_m"),

    # Estandarizar manualmente: (valor - media) / desviación estándar
    (
        (pl.col("altura") - pl.col("altura").mean()) /
        pl.col("altura").std()
    ).round(2).alias("altura_z"),
])
```

Dentro de `with_columns`, `.mean()` y `.std()` calculan el escalar para toda la columna y lo asignan a cada fila. En la próxima sección veremos cómo calcular esas medidas por grupo con `.over()`.

`pl.concat()`

Apila dos o más DataFrames verticalmente. Equivale a un `UNION ALL` en SQL:

```
# Esquemas idénticos: usar how="vertical" (por defecto)
df_alumnos_completo = pl.concat([df_alumnos, df_new_alumnos])

# Esquemas distintos (columnas no comunes): how="diagonal" rellena con null
df_asistencia_completa = pl.concat(
    [df_asistencia, df_new_asistencia],
    how="diagonal",
)
```

`how="vertical"` falla si los esquemas no coinciden exactamente. `how="diagonal"` acepta columnas distintas y rellena con `null` donde no hay datos. Verificar el esquema con `set` antes de decidir cuál usar.

`pl.concat()` no elimina duplicados. Si un registro aparece en ambas tablas, quedará dos veces en el resultado. Para eliminar duplicados después: `.unique()`.

Reshaping

Cambiar la forma de la tabla: wide y long

Pivot wide: de long a wide

La tabla `notas` tiene una fila por alumno por materia (formato **long**). `.pivot()` la convierte a **wide** para tener una columna por materia:

```
notas_wide = notas.pivot(  
    on="materia",           # columna cuyos valores se vuelven encabezados  
    index=["cod_estudiante", "nombre"], # columnas que identifican cada fila  
    values="nota",         # columna que rellena las celdas  
    aggregate_function="first",  
)
```

cod_estudiante	nombre	CASTELLANO	EDUCACION FISICA	MATEMATICAS	TEATRO
1	Raul Martos	0.0	10.0	3.73	1.86
2	Lazaro Herrera	6.89	9.47	0.38	10.0

Pivot long: de wide a long

La tabla `asistencia` tiene una columna por día (formato **wide**). `.unpivot()` la convierte a **long** para tener una fila por alumno por día:

```
asist_long = asistencia.unpivot(  
    on=pl.selectors.starts_with("a_"), # columnas que se convierten en filas  
    index=["cod_estudiante", "nombre"], # columnas que se repiten en cada fila  
    variable_name="fecha",  
    value_name="asistio",  
)
```

El resultado pasa de **20 filas x 23 columnas** a **460 filas x 4 columnas**: una fila por alumno por día.

En formato wide, filtrar "todos los días en que asistió Andreea" requeriría 23 condiciones. En formato long basta con `.filter(pl.col("nombre") == "Andreea Centeno")`.

Joins

Combinar tablas por claves comunes

Tipos de join

Tipo	¿Qué conserva?	Cuándo usarlo
left	Todas las filas de la tabla izquierda	Agregar columnas sin perder filas base
inner	Solo filas con coincidencia en ambas tablas	Subconjunto garantizado en las dos tablas
full	Todas las filas de ambas tablas	Diagnóstico: ver qué falta en cada lado
semi	Filas izquierda con coincidencia, sin columnas nuevas	Filtrar por existencia en otra tabla
anti	Filas izquierda sin coincidencia	Encontrar registros sin pareja

Antes de cualquier join: verificar que la llave no tiene duplicados en la tabla derecha. Un duplicado convierte un left join en una multiplicación silenciosa de filas.

Full join: diagnóstico de cobertura

El full join conserva **todas las filas de ambas tablas**. Las filas sin pareja quedan con `null` en las columnas del otro lado:

```
df_todos = (  
    df_alumnos_orig  
    .join(df_new_alumnos, on="cod_estudiante", how="full", suffix="_nuevo")  
    .with_columns(  
        pl.when(pl.col("cod_estudiante_nuevo").is_not_null())  
        .then(pl.lit(True))  
        .otherwise(pl.lit(False))  
        .alias("nuevo_estudiante")  
    )  
)
```

El parámetro `suffix` evita colisiones de nombre: la columna llave de la tabla derecha se llama `cod_estudiante_nuevo`. Si ese campo es `null`, la fila solo existe en la tabla izquierda; si el original es `null`, solo existe en la derecha.

Semi join y anti join

Filtran filas de la tabla izquierda **sin agregar columnas** nuevas:

Semi join: conserva las filas que **tienen** pareja

```
# Solo alumnos que aparecen en resumen_asistencia
df_viaje_notas.join(
    asistencia_cumple,
    on="cod_estudiante",
    how="semi",
)
```

Equivale a: "quédate con las filas de la izquierda que existen en la derecha."

Anti join: conserva las filas que **no tienen** pareja

```
# Alumnos que NO están en la lista de baja asistencia
df_viaje_notas.join(
    asistencia_no_cumple,
    on="cod_estudiante",
    how="anti",
)
```

Equivale a: "quédate con las filas de la izquierda que no existen en la derecha."

A diferencia del inner join, el semi join no duplica filas aunque la tabla derecha tenga múltiples coincidencias. Y a diferencia del left join, no agrega columnas: el resultado tiene exactamente las mismas columnas que la tabla izquierda.

Normalizar antes del join

Definimos una función reutilizable que aplica todas las transformaciones de texto necesarias:

```
def normalizar_materia(expr: pl.Expr) -> pl.Expr:
    return (
        expr.str.to_lowercase()
            .str.strip_chars()
            .str.replace_all("á", "a")
            .str.replace_all("é", "e")
            .str.replace_all("í", "i")
            .str.replace_all("ó", "o")
            .str.replace_all("ú", "u")
    )

notas_norm = notas.with_columns(
    normalizar_materia(pl.col("materia")).alias("materia")
)
prof_norm = profesores.with_columns(
    normalizar_materia(pl.col("materia")).alias("materia")
)
# Ahora "MATEMATICAS" y "matemáticas" quedan ambas como "matematicas"
```

Join de cuatro tablas

Tabla auxiliar: qué profesor enseña cada materia en cada paralelo

```
prof_paralelo = (  
    asigna_clase  
    .join(prof_norm, on="cod_profesor", how="left")  
    .rename({"nombre": "nombre_profesor"})  
    .select(["paralelo", "materia", "nombre_profesor"])  
)
```

Join encadenado: notas -> alumnos -> prof_paralelo

```
resultado = (  
    notas_norm  
    .join(  
        alumnos.select(["cod_estudiante", "paralelo"]),  
        on="cod_estudiante",  
        how="left",  
    )  
    .join(  
        prof_paralelo,  
        on=["paralelo", "materia"],  
        how="left",  
    )  
)
```

Agrupaciones

group_by y agg para resumir por grupos

group_by por profesor

Promedio de notas que asignó cada profesor en sus materias y paralelos:

```
por_profesor = (  
    resultado  
    .group_by("nombre_profesor")  
    .agg([  
        pl.col("nota").mean().alias("promedio"),  
        pl.col("nota").count().alias("n_notas"),  
    ])  
    .sort("promedio", descending=True)  
)
```

`pl.col("nota").count()` cuenta solo valores no nulos. `pl.len()` contaría todas las filas del grupo, incluyendo los nulls. En este caso la diferencia importa porque dos alumnos no tienen calificación en ninguna materia.

group_by por alumno

Promedio general de cada estudiante a lo largo de todas las materias:

```
por_alumno = (  
    resultado  
    .group_by(["cod_estudiante", "nombre"])  
    .agg([  
        pl.col("nota").mean().alias("promedio_general"),  
        pl.col("nota").count().alias("materias_evaluadas"),  
    ])  
    .sort("promedio_general", descending=True)  
)
```

Cuando la llave de agrupación tiene más de una columna, todas deben ir en una lista. Aquí incluimos `cod_estudiante` y `nombre` porque hay pequeñas inconsistencias en los nombres entre tablas: la llave numérica garantiza la identidad del alumno.

Window functions

Calcular agregados por grupo sin reducir las filas

Z-score por materia con .over()

`.over()` calcula un agregado particionado por grupo y lo asigna a cada fila. El número de filas no cambia. Es el equivalente de `PARTITION BY` en SQL:

```
df_zscore = resultado.with_columns([
    pl.col("nota").mean().over("materia").alias("media_materia"),
    pl.col("nota").std().over("materia").alias("std_materia"),
]).with_columns(
    ((pl.col("nota") - pl.col("media_materia")) / pl.col("std_materia"))
    .round(2)
    .alias("z_score")
)
```

group_by - reduce filas

materia	promedio
castellano	2.81
matematicas	6.87

over - mantiene todas las filas

nombre	materia	nota	media	z_score
Raul Martos	castellano	0.0	2.81	-1.09
Santos Rico	castellano	2.27	2.81	-0.21

¿Qué aprendimos hoy?

Conceptos

- Ingesta desde CSV, Excel y R como pasos iniciales del ETL
- `with_columns` agrega sin eliminar y opera en bloque
- Normalizar texto antes de un join evita errores silenciosos
- Long es para calcular, wide es para presentar
- `group_by` reduce filas; `.over()` mantiene todas las filas

En Polars

- `pl.read_csv()` con `separator` y `encoding` no estándar
- `.rename()` con diccionario y con lambda
- `set()` para comparar esquemas entre tablas
- `pl.concat()` con `how="vertical"` y `how="diagonal"`
- `str.contains()` con patrones regex
- `.join()` con `how="full"`, `how="semi"`, `how="anti"`
- `.group_by().agg()`, `.over()`